



PDF Download
3764727.3764733.pdf
03 March 2026
Total Citations: 0
Total Downloads: 166

Latest updates: <https://dl.acm.org/doi/10.1145/3764727.3764733>

RESEARCH-ARTICLE

Stock Price Prediction Method Based on Optimized LSTM and Quantified News Sentiment

CHENGSHANG WU, Dalian Maritime University, Dalian, Liaoning, China

ZHIHENG SONG, South China University of Technology, Guangzhou, Guangdong, China

ZHIQI NIE, Sun Yat-Sen University, Guangzhou, Guangdong, China

YUNUO WU, Sun Yat-Sen University, Guangzhou, Guangdong, China

JIFAN ZOU, Xiamen University, Xiamen, Fujian, China

Open Access Support provided by:

Sun Yat-Sen University

Xiamen University

Dalian Maritime University

South China University of Technology

Published: 20 November 2025

Citation in BibTeX format

AIDF 2025: 2025 International Conference
on Artificial Intelligence and Digital
Finance

June 27 - 29, 2025
Chengdu, China

Stock Price Prediction Method Based on Optimized LSTM and Quantified News Sentiment

Chengshang Wu*
International Joint College
Dalian Maritime University
Dalian, Liaoning, China
15973151776@163.com

Zhiheng Song
School of Future Technology
South China University of
Technology
Guangzhou, Guangdong, China
202330841441@mail.scut.edu.cn

Zhiqi Nie
Business School
Sun Yat-sen University
Shenzhen, Guangdong, China
niezhq@mail2.sysu.edu.cn

Yunuo Wu
Software engineering
Sun Yat-sen University
Zhuhai, Guangdong, China
wuyun65@mail2.sysu.edu.cn

Jifan Zou
School of informatics
Xiamen University
Xiamen, Fujian, China
37220222203904@stu.xmu.edu.cn

Abstract

Predicting stock prices is a challenging task influenced by various factors, including historical trends and market sentiment. Traditional time series models, such as ARIMA, and conventional machine learning models, like Support Vector Machines (SVM) and K-Nearest Neighbors (KNN), often struggle to capture the nonlinear characteristics of the stock market, resulting in low forecasting accuracy [1]. This paper introduces a hybrid model that combines Long Short-Term Memory (LSTM) networks optimized with Genetic Algorithms (GA) and text sentiment analysis using pre-trained language models based on BERT and the cnsenti package. This approach effectively captures features from multimodal data, leading to improved predictions for Nvidia's stock price. To enhance the model's ability to learn from stock splits, we apply a price adjustment technique for continuity. The LSTM model, based on historical data, predicts stock prices, while the sentiment analysis model extracts insights from financial news. We utilize a weighted fusion approach to combine these predictions, providing a more comprehensive outlook on stock price movements. The final evaluation includes multiple performance metrics, such as Mean Squared Error (MSE), R-squared, Mean Absolute Error (MAE) and compares our model with general LSTM. Experimental results show that our hybrid method outperforms traditional LSTM models in terms accuracy and precision, offering a more robust framework for stock price forecasting.

CCS Concepts

• Computing methodologies; • Artificial intelligence; • Natural language processing; • Information extraction;

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. AIDF 2025, Chengdu, China

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1495-5/2025/06
<https://doi.org/10.1145/3764727.3764733>

Keywords

GA-LSTM, News Sentiment Quantification, Quantitative Investment

ACM Reference Format:

Chengshang Wu, Zhiheng Song, Zhiqi Nie, Yunuo Wu, and Jifan Zou. 2025. Stock Price Prediction Method Based on Optimized LSTM and Quantified News Sentiment. In *2025 International Conference on Artificial Intelligence and Digital Finance (AIDF 2025)*, June 27–29, 2025, Chengdu, China. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3764727.3764733>

1 INTRODUCTION

The stock market holds a crucial position in the global economic landscape. It acts as an indicator of financial well-being and a key platform for wealth creation. Predicting the stock market has been a subject of great interest in both economic circles and academia for a long time. However, Precise stock price prediction is extremely challenging yet of utmost importance for investors, policymakers, and financial institutions. Given the highly volatile and non-linear characteristics of stock prices, conventional statistical methods such as the traditional econometric models ARIMA and GARCH frequently struggle to identify complex patterns. With the emergence of machine learning and deep learning, certain models have shown good performance in capturing the intricate features of the stock market. For example, in the research of S. Selvin et al [2], the combination of LSTM, RNN, and CNN has effectively captured temporal dependencies, thus achieving remarkable progress in time-series forecasting.

Nevertheless, stock prices are affected not only by historical trends but also by external elements. These include investor sentiment, significant news regarding relevant market events, and macroeconomic indicators. Previous studies have indicated that market sentiment, which can be derived from news articles, social media, or analyst reports, has a substantial impact on stock volatility and price fluctuations. For instance, Sousa et al [3] applied the BERT method to analyze stock prices and found that online discussions are correlated with trading volume and stock returns. Meanwhile, Weng et al [4]. integrated BERT-derived sentiment scores with LSTM to boost the prediction accuracy of Chinese stocks. Despite these advancements, many existing models

either neglect sentiment analysis or are ineffective in optimizing hyperparameters, resulting in sub-par performance.

So to deal with the problems we mentioned earlier, we came up with kind of a two-way method. Basically, we mix past stock price info with stuff like news and online text to help predict where the market’s going. First, we built a LSTM model that gets tuned by this thing called a genetic algorithm (GA) – it’s like nature-inspired optimization. It helps us pick better hyperparameters, so the predictions end up more accurate. Oh, and we also added a price adjustment step to handle situations like stock splits – that way, the model can keep learning smoothly and won’t get confused by sudden price jumps.

Then on the other side, we used a few BERT-based models – like Chinese-BERT-wwm-ext and bert-base-multilingual-uncased (these two are better suited for Chinese-language news), plus a Python package called cnsenti. These tools help us pull out how people feel from financial news, which gives some clues about market moods and future trends. After we get both types of predictions – one from LSTM, one from sentiment – we combine them using a weighted fusion (fancy way of mixing) to make the final prediction. We tested everything using several ML metrics to see how well the whole thing works. Overall, we wanted a more all-around and accurate way to guess stock prices, something that helps both academic researchers and real-life investors.

By putting GA-tuned LSTM together with BERT-based sentiment analysis, our method’s pretty good at handling stocks that jump around a lot – like NVIDIA and others. Since it looks at both the numbers (like price changes) and the vibes (what people are saying online or in news), it actually beats regular LSTM and a bunch of old-school machine learning stuff. This whole system gives traders and fund managers some real-world insights by showing how news or mood might actually affect the price swings.

2 RELATED WORK

2.1 The Relationship between News Sentiment Analysis and Stock Prices

Although historical price data has long been a primary input into stock forecasting models, relying purely on past trends often fails to provide accurate forecasts, especially during turbulent market periods or when unexpected events occur. To address this issue, the relationship between news sentiment and stock prices has been increasingly explored in research in recent years [5]. News reports can influence investors’ behavioral decisions, which in turn affect stock prices. Stock market investors seek higher profits by analyzing market information such as financial news [6], so sentiment analysis of these financial news can better predict stock prices by combining sentiment indicators of the news. Traditional sentiment analysis methods are usually based on sentiment dictionaries, such as the Loughran-McDonald sentiment dictionary, which classifies words into positive, negative, and neutral sentiments and scores them based on the frequency of occurrence of different types of words in the article [7]. However, this approach has limitations in capturing the nuances of natural language. On the other hand, modern sentiment analysis techniques leverage advanced natural language processing (NLP) models, such as recurrent Neural Networks (RNN), BERT (Bidirectional Encoder Representation from

Transformers), and GPT (Generation of Pre-trained Transformer) models to achieve more accurate sentiment classification [8]. These models have made significant progress in sentiment analysis tasks and are increasingly being used in financial predictive modeling..

2.2 Combining Sentiment Analysis with Financial Modeling

Based on the established relationship between news sentiment and stock prices, some studies have already explored how to enhance traditional financial models using sentiment features extracted from news articles and social media. For example, Jing et al. used a CNN model to classify investor sentiment in major Chinese stock forums and used the LSTM model to build a stock price prediction model to demonstrate that the hybrid model with sentiment outperforms the baseline model [9]. In addition, Sousa et al. used the bidirectional encoder representation in the Transformer architecture like BERT to perform sentiment analysis on stock news articles [10].

2.3 Challenges and Future Directions

Although current research has made great progress in combining sentiment analysis with machine learning for stock price prediction, some challenges still remain. For example, the continuous changes in financial markets require models that can adapt to changing market dynamics and effectively integrate real-time data [11]. Future research directions may include developing more advanced sentiment analysis models to process complex financial text data, exploring the impact of different types of news on stock prices, and designing adaptive models that can learn from continuous data streams.

In summary, the combination of sentiment analysis and machine learning techniques has been shown to be effective in improving the accuracy of stock price predictions. This proposal aims to use advanced natural language processing models and machine learning algorithms to grasp the temporal characteristics of stock prices and sentiment trends in financial news based on existing research, thereby achieving more accurate stock price predictions.

3 METHODOLOGY

Our approach uses fusion data to enhance stock price forecasting. The methodology includes how we extract the news sentiment feature, how the LSTM architecture works, and how can GA find the best set of hyperparameters.

3.1 News Sentiment Feature Extraction

After acquiring the news sentiment data, three methods below are used to get the news sentiment feature.

3.1.1 Chinese-BERT-wwm-ext. The model we’re using is basically BERT – a language model that’s really good at “understanding context.” The process is pretty straightforward: we pull out the daily news from an Excel file, then use BERT’s tokenizer to break each piece of news into tokens. After that, we feed the news into the model, which analyzes the overall meaning of the sentence. Finally, it goes through a classification step to give us the probability of whether the news is more positive or negative, and we take the

positive score as the sentiment value.

$$p = \text{softmax}(W \cdot h_{[CLS]} + b) \quad (1)$$

Here, $h_{[CLS]}$ denotes the contextualized sentence embedding, and W, b are the parameters of the classification layer. The resulting probabilities $p = [p_0, p_1]$ correspond to negative and positive sentiment scores, respectively. We retained the positive sentiment score p_1 as the final sentiment feature for each news item. These scores were appended to the original dataset to serve as sentiment inputs for subsequent stock price prediction models.

3.1.2 Bert-base-multilingual-uncased. We're using a BERT-based multilingual classifier here. It was originally built to rate sentiment on a 1-to-5 star scale, but we tweaked it so it gives us a continuous score from -10 to 10, which works better for financial stuff. What we do is load up an Excel file that has the daily news, clean up the data by fixing the date formats and getting rid of anything blank. Then we break the news text into tokens using HuggingFace's BERT tokenizer and run those through a pre-trained BERT model. The model spits out some raw values — called logits — and we turn those into probabilities with softmax. These probabilities tell us how likely the news is negative, neutral, or positive. In the end, we calculate a single sentiment score using a weighted formula that maps strong negativity to -10, strong positivity to +10, and neutral stuff to 0.

$$\text{Sentiment_Score} = -10 \cdot p_1 + 0 \cdot p_2 + 10 \cdot p_3 \quad (2)$$

This equation transforms class probabilities into a continuous sentiment value, where strong negative sentiment maps to -10, strong positive sentiment to +10, and neutral sentiment to 0, providing a fine-grained measure of emotional intensity.

3.1.3 Cnsenti. Cnsenti is this handy tool for analyzing sentiment in Chinese text using a built-in dictionary. It comes with lists of positive words, negative words, negation words, and also degree adverbs, and it's pretty good at quickly counting up sentiment-related stuff. In our case, we cleaned up the news data and pulled out two text columns (text1 and text2). For each one, we figured out the sentiment score just by taking the number of positive words and minus the number of negative ones. Here's the basic formula we used:

$$\text{Sentiment_Score} = N_{pos} - N_{neg} \quad (3)$$

N_{pos} is the number of positive sentiment words in the text; the number of negative sentiment words is represented by N_{neg} . This rule-based approach does not require model training, so it is particularly suitable for resource-poor scenarios and has high interpretability in financial applications. The resulting sentiment scores are denoted as sentiment1 (from text 1) and sentiment2 (from text 2), stored in the output file, and subsequently used as input features for our stock price prediction model.

3.2 LSTM Architecture

After extracting the sentiment features from the news dataset, we are expected to integrate these feature into a predictive model which can capture the temporal dependencies relationship. The model we used is Long Short-Term Memory (LSTM).

3.2.1 Bidirectional LSTM (BiLSTM). In this study, a Bidirectional Long Short-Term Memory (BiLSTM) network is used to better capture both forward and backward temporal dynamics within the stock price sequences. The quantity of hidden units within each BiLSTM layer, represented by the hyperparameter u , is automatically optimized using a Genetic Algorithm.

The BiLSTM structure comprises two LSTM layers: one that analyzes the input sequence from earlier to later stages (forward direction), while the other processes it in reverse (backward pass). For every time step in the input of shape (t, f) , where t signifies the time step and f represents the feature dimension, the forward and backward hidden states are concatenated to form a comprehensive representation:

$$h_t = \vec{h}_t \oplus \overleftarrow{h}_t \quad (4)$$

This bidirectional setup can make full use of the context information. Particularly, it shows great effect in financial time series such as delayed responses and reversals in trends. Our model uses two BiLSTM layers to capture the features of deep series, we also imply a Dropout after each layer to deal with the overfitting.

3.2.2 Dropout Mechanism and Hyperparameter Optimization. The dropout rate is controlled by the hyperparameter d , which is also optimized by the Genetic Algorithm within the range $[0, 0.5]$. During training, Dropout randomly "deactivates" a fraction d of the neurons, effectively training a different subnetwork at each iteration and thereby enhancing the model's ability to generalize. The Dropout operation is expressed as:

$$\tilde{h}_t = \text{Dropout}(h_t, d) \quad (5)$$

where h_t is the output of the BiLSTM layer and $\tilde{h}_t$ is the regularized feature vector. The Genetic Algorithm searches for the optimal dropout rate d by minimizing the Mean Squared Error (MSE) on the validation set, thereby balancing model fit and generalization.

3.3 Genetic Algorithm for Hyperparameter Optimization

As Figure 1 below, in order to improve the generalization ability of the model, we use Genetic Algorithm (GA) to search the best combination of hyperparameter automatically [12]. we aim to get the MSE through iterative evolution over a defined hyperparameter space.

3.3.1 Problem Formulation. The hyperparameter optimization problem is formulated as follows:

$$\min_{\theta \in \Theta} \text{MSE}_{val}(f(X_{train}; \theta), y_{val}) \quad (6)$$

where $\theta = [u, t, \eta, b, d]$ represents the hyperparameter set, in which u denotes the quantity of LSTM units, t represents the input time step length, η represents the learning rate, b is the batch size, and d indicates the dropout rate. The objective is to find the hyperparameter configuration θ that minimizes the validation MSE after training the model.

3.3.2 Individual Encoding and Initialization. Each "individual" in our population basically holds a full set of hyperparameters, written out as a vector of real numbers. To get things started, we randomly create the first batch of these individuals by picking values from

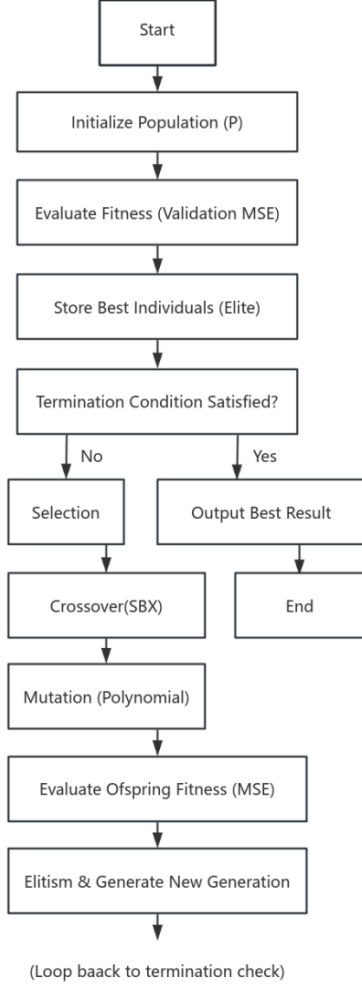


Figure 1: Brief Genetic Algorithm model architecture.

preset ranges — some are fixed options, others are continuous. For example:

Units: [32,64,96,128,144], Time steps: [1,4,8,12,16], Learning rate: continuous in [0.0001,0.0011], Batch size: [16,32,...,112], Dropout: continuous in [0.15,0.45].

3.3.3 Fitness Evaluation. The fitness of each individual is determined by training the model with the encoded hyperparameters and calculating the MSE on the validation set. The fitness function is as follows:

$$\text{Fitness}(\theta) = \text{MSE}_{\text{val}} \quad (7)$$

Lower MSE indicates better performance.

3.3.4 Genetic Operators. We utilize standard genetic operators to evolve the population across generations. One is Simulated Binary Crossover (SBX), This operator imitates single-point crossover in binary representation and generates offspring around the parents with a controlled spread defined by the distribution index η_c . It

is applied with a probability $p_c=0.65$. The other is Polynomial Mutation, A bounded mutation operator that perturbs individuals slightly within their variable bounds using a polynomial probability distribution with index η_m . Mutation probability is set to $p_m=0.05$.

3.3.5 Elitism and Population Evolution. To make sure we don't lose the best solutions, we use something called an elite strategy. Basically, in every generation, we keep the top 5 individuals based on how well they perform. Then, we take the 3 worst new ones and swap them out with the top 3 elites. The other 2 elites just get added in directly. To keep the total number the same, we kick out the two lowest-performing individuals. This way, even if crossover or mutation messes things up a bit, our best configurations stay safe.

3.3.6 Evolution Process and Termination. The population evolves over $G=10$ generations, with a population size of $N=45$. In each generation, the average and minimum MSE are logged for performance tracking. Upon completion, the best individual represents the optimal hyperparameter configuration.

4 EXPERIMENT

4.1 Data Set

This paper integrates NVIDIA stock price data and Chinese financial news sentiment data for stock closing price prediction:

Stock Price Data: Historical open, high, low, close prices, and trading volume from September 12, 2019, to March 12, 2025, obtained through the python package akshare.

News Data: 3474 Chinese news from National Business Daily (NBD, a leading financial media platform in China), from March 1, 2020, to March 12, 2025. Each includes headlines, publication dates, and full text.

Although the two datasets have different initial time spans, in Feature Engineering section will align the time spans and ensure consistency between the stock price data and news sentiment data.

4.2 Data Preprocessing

This paper checks that there are no missing values in both data sets. The stock data contained duplicate values (Open:15, Close:21, High:19, Low:16), while the news data set has no duplicate values. However, as stock price, duplicate values is normal so this paper does not dispose them. Similarly, stock price outliers should not be disposed. Except missing values, stock split is another factor need to be considered when process data. As can be seen in Figure 2 below, NVIDIA has split stock twice during the 5 years period. This completely artificial factor will cause a sudden.

change in stock prices and greatly interfere with the learning of the model [13]. Therefore, this paper use Equation (8) to adjust stock price and Equation (9) to adjust volume:

$$P_t^{(k)} \begin{cases} \frac{P_t^{(k-1)}}{S_k}, & t < d_k \\ P_t^{(k-1)}, & \text{otherwise} \end{cases} \quad (8)$$

$$V_t^{(k)} \begin{cases} V_t^{(k-1)} \times S_k, & t < d_k \\ V_t^{(k-1)}, & \text{otherwise} \end{cases} \quad (9)$$

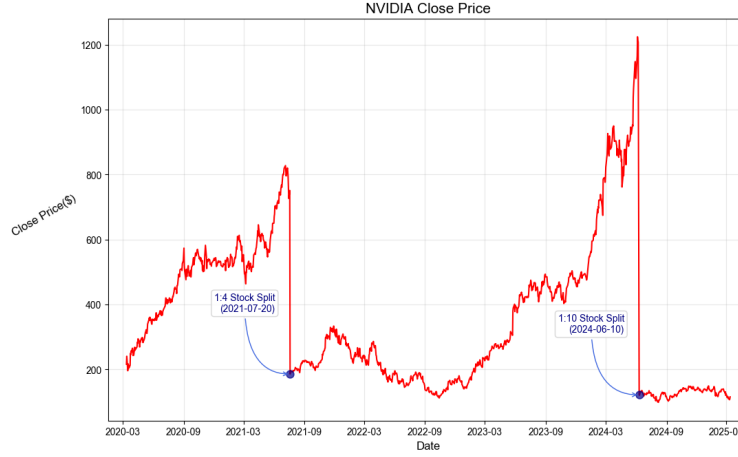


Figure 2: Original closing price of NVIDIA.

Table 1: Top 10 indicators of correlation with closing price.

Indica- tors:	Open	Volatility dch	Volatility bbh	Trend visual ichimoku a	Momentm kama	volatili- tydcl	high	low	others cr	volume adi
Scores:	13024.18	3250.74	3006.72	2748.73	1269.87	854.37	751.85	750.58	648.63	476.81

where $P_t^{(k)}$ is the price (close/open/high/low) at time t after adjusting for the k -th split; $P_t^{(0)}$ is unadjusted prices; s_k is the split ratio; d_k is the Effective date of the k -th split; $V_t^{(k)}$ is Trading volume at time t after adjusting for the k -th split; $V_t^{(0)}$ is unadjusted volumes.

4.3 Feature Engineering

After data preprocessing, this paper conducts feature engineering to improve the prediction accuracy of the model. This paper uses four basic features from stock data (Open, High, Low, Volume) and generates 86 technical indicators via the python package ta. To solve early-stage missing values caused by calculation windows, this paper uses complete data from March 1, 2020, to March 12, 2025. Then this paper uses XGBoost to select some highly correlation indicators with closing price (See Appendix for selecting principles). Table 1 shows the top 10 indicators of correlation with closing price. Based on Table 1, this paper selects “volatility dcl”, “volatility bbh”, “trend visual ichimoku a”, and “others cr” as the features added into stock price data set. However, the trend visual ichimoku a has risk in data leakage roughly because of backward-shifting of the Ichimoku cloud so this paper uses the similar indicators trend ichimoku a to replace it (See appendix for detailed formula and risk analysis).

4.4 GA-NEWS-LSTM Model

The brief GA-NEWS-LSTM model architecture is given in Figure 3 below. The following paper explains each part in detail.

4.4.1 Stock Price Data Normalization. Data normalization is an important step before neural network training. The clean stock price data has been obtained. It has 10 features, which is date,

open, close, high, low, volatility dcl, volatility bbh, trend ichimoku a and others cr.

This paper uses Min-Max Normalization given by Equation (10) to normalize features except date:

$$x_{norm} = \frac{x - \min(X)}{\max(X) - \min(X)} \quad (10)$$

In Equation (10), x_{norm} is the normalized feature value; x is the original feature value; $\min(X)$ is the minimum value of feature X ; $\max(X)$ is the maximum value of feature X .

Because the stock data set is time series data, in order to avoid time information leakage, the data set is partitioned into time order. In this paper, the first 80% is the training set and the last 20% is the test set. The test set contains closing price while training set contains other 8 features excluding date.

4.4.2 Natural Language Processing of News Data. News sentiment scores reflect the market environment and investor sentiment, which is very helpful for stock price prediction. This paper uses the three different natural language processing methods mentioned in the research methods to perform sentiment scoring on the news data, and normalize the three scores using Equation (10) and merge them with the stock price data obtained in 4.4.1 to obtain the fusion data set.

4.4.3 News Sentiment Dynamic Weighting. News sentiment dynamic weighting improves model’s ability of highlighting important features, generalization, and adapting to dynamic changes in data. This paper implements dynamic weighting of news sentiment through three steps: news feature dimension increase, news feature weight matrix generation and fusion. The following is a detailed illustration:

News Feature Dimension Increase. Firstly, Extract news sentiment features normalized by Equation 10 from the fused data. Secondly, create a Dense layer that is a fully connected neuron layer and linearly combined input news sentiment features using formula given by Equation (11):

$$Z = W_e \cdot S + b_c \quad (11)$$

where Z is intermediate result after linear transformation; W_e is weight matrix of Dense; S is input sentiment features; b_c is bias term of Dense layer.

Thirdly, apply nonlinear activation Swish input function given by Equation (12):

$$S' = Z \odot \sigma(\beta Z) \quad (12)$$

where S' is the result after nonlinear transformation; Z is given in Equation 4; $\odot$ means Hadamard Product; β is learnable parameter; σ is Sigmoid function given by Equation (13):

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (13)$$

Finally, puts S' which given in Equation (12) in Dense layer and sets the target dimension to half the number of LSTM hidden units. The dimension of news sentiment have been increased.

News Weight Matrix Generation. Create a Dense layer and generate new weights matrix using the formula given by Equation (14):

$$\alpha = \sigma(S' \cdot W_a + b_a) \quad (14)$$

where α is the news weight matrix; σ is the Sigmoid function given by Equation (13); S' is given in Equation (12); W_a is weight matrix of Dense; b_a is bias term of Dense.

Fusion. Use the Equation (15) to obtain the weighted news sentiment features:

$$\tilde{S} = S' \odot \alpha \quad (15)$$

where $\tilde{S}$ is the weighted news sentiment features; S' is given in Equation (12); $\odot$ means Hadamard Product; α is given in Equation (14).

4.4.4 Gating Mechanism. This paper uses a gating mechanism that can not only solve the gradient vanishing problem of deep neural networks, but also adaptively adjust the contribution of features to closing price prediction according to the gating weight matrix. This paper implements the gating mechanism in two steps: gating weight matrix generation and Modulation of features of the stock price data. The following is a detailed illustration:

Gating Weight Matrix Generation. Firstly, this paper create a Dense layer then extracts the 8 features of stock price data from the fused data. Next obtains the gating weight matrix using formula given by Equation (16):

$$G_{t,j} = \sigma\left(\sum_{k=1}^{dim} W_g^{(j,k)} \cdot \tilde{S}_{t,k} + b_g^{(j)}\right) \quad (16)$$

where $G_{t,j}$ is the importance of the j -th features of the stock price data at the t -th time step; σ is the Sigmoid function given by Equation (13); dim is the target dimension set in 4.4.3.1; $W_g^{(j,k)}$ is the connection weight of Dense layer of the j -th gate unit to the k -th dimension sentiment feature; $\tilde{S}_{t,k}$ is the weighted news sentiment features which is given in Equation (15) of the k -th dimension at

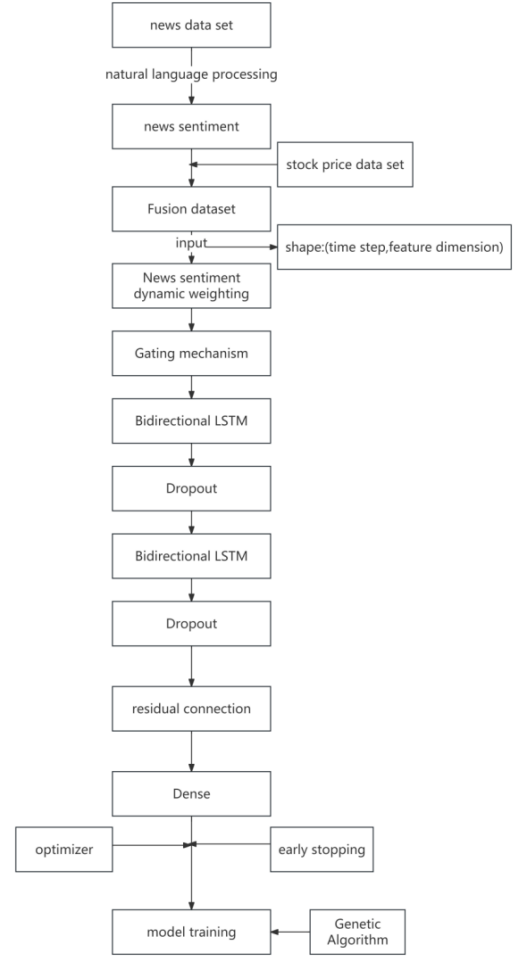


Figure 3: Brief GA-NEWS-LSTM model architecture.

the t -th time step; $b_g^{(j)}$ is the bias term of Dense layer of the j -th gated unit.

Modulation of Features of The Stock Price Data. This paper modulate the features using formula given by Equation (17):

$$\tilde{X}_{t,j} = X_{t,j} \cdot G_{t,j} \quad (17)$$

where $\tilde{X}_{t,j}$ is modulated features; $X_{t,j}$ is the j -th raw features value at the t -th time step; $G_{t,j}$ is given in Equation (16).

4.4.5 LSTM Architecture. Before building LSTM, it first need to fuse the weighted news sentiment features with the modulated stock price data features. This paper fuse them using formula given by Equation (18):

$$C_t = [\tilde{X}_t || \tilde{S}_t] \quad (18)$$

where C_t is the fused feature vector at the t -th time step; $\tilde{X}_t$ is the modulated stock data features vector at the t -th time step and $\tilde{S}$ is given in Equation (17); $||$ is concatenate along the feature dimension to merge two vectors into a joint feature vector; $\tilde{S}_t$ weighted news

sentiment feature vector at the t -th time step and $\tilde{S}_t$ is given in Equation (15).

Then the C_t is input to bidirectional LSTM layers with hidden units, whose number will be determined by the subsequent genetic algorithm. And so do the another bidirectional LSTM layers. The dropout rate of two dropout layers will also be determined by the genetic algorithm. After two layers of bidirectional LSTM and two layers of Dropout, the final hidden state is obtained and is set to h_{LSTM} .

In order to prevent the information attenuation of deep networks and alleviate the gradient vanishing problem, this paper used a residual connection before the output layer. Firstly create a Dense layer and use the formula by Equation (19):

$$h_{cut} = W_s \cdot C_T + b_s \quad (19)$$

where h_{cut} is the vector of the residual path; W_s is weight matrix of Dense layer; C_T is the fused features of the last time step of the time window; b_s is bias term of Dense layer. Secondly concentrate h_{LSTM} with h_{cut} using formula given by Equation (20):

$$h_{final} = [h_{LSTM} || h_{cut}] \quad (20)$$

where h_{final} is concentration vector; h_{LSTM} is the final hidden state; $||$ is concatenate along the feature dimension to merge two vectors into a joint feature vector; h_{cut} is given in Equation (19). Finally, create Dense layer and obtain the output prediction values using formula given by Equation (21):

$$\hat{y} = W_o \cdot h_{final} + b_o \quad (21)$$

where $\hat{y}$ is the the output prediction values; W_o is weight matrix of Dense layer; h_{final} is given in Equation (20); b_o is bias term of Dense layer.

As for the optimizer, this paper used Adam with initial learning going to be determined by the genetic algorithm and for the loss function, this paper uses the mean square error given by Equation (22):

$$\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (22)$$

where N is the amount of samples; y_i is the true value; $\hat{y}_i$ is the model prediction value.

When training the model, this paper sets 10 as the maximum number of training epochs and the batch size will be determined by the genetic algorithm. Meanwhile, 20% of the training set is divided as the validation set, and the remaining 80% is used as the training set. And this paper uses an early stopping mechanism. If validation loss has no improvement for 3 consecutive epochs, the training will terminate and the model will roll back to the state with the minimum validation loss.

4.4.6 Genetic Algorithm. This paper uses the genetic algorithm given in the methodology to find a better combination of hyperparameters. The hyperparameters this paper are looking for include the number of units in the bidirectional LSTM layer, the dropout rate of the Dropout layer, the batch size during training, the initial learning rate, and the time step of the input layer. Table 2 shows the hyperparameter combinations obtained by genetic algorithm. Then pass the hyperparameter combination settings to the LSTM

architecture given in 4.4.5 to train the model and obtain the closing price predicted values.

4.5 GA-NEWS-LSTM Model Performance Evaluation

This paper uses three common regression task indicators (R^2 , MSE and MAE) to evaluate our proposed A-NEWS-LSTM and compares its performance with that of ordinary LSTM.

Figure 4 shows the comparison of closing price prediction values of GA-NEWS-LSTM and actual values while Figure 5 shows the shows the comparison of closing price prediction values of traditional LSTM and actual values.

And Table 3 shows the comparison between GA-NEWS-LSTM and LSTM under common regression evaluation indicators. The formula of R^2 is given by Equation (23):

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (23)$$

$$\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (24)$$

where n is the number of samples; y_i represents the true value of the i -th sample; $\hat{y}_i$ represents the predicted value of the i -th sample; $\bar{y}$ represents the mean of the true values. The closer R^2 is to 1, the better the model fit is. And the formula of MSE is given by Equation (22). The formula of MAE (mean absolute error) is given by Equation (24) where the meaning of symbols is consistent with Equation (23).

5 CONCLUSION

We present an integrated deep learning model designed for predicting stock prices through sentiment analysis and historical stock data. We employ a Genetic Algorithm-Long Short-Term Memory (GA-LSTM) framework to optimize hyperparameters for both sentiment analysis and time series forecasting. The optimized model shows excellent performance, as evidenced by lower Mean Absolute Error (MAE) and Mean Squared Error (MSE), in addition to an improved R^2 score. Although the model occasionally lags in forecasting stock price trends, this limitation is deemed acceptable given its lightweight and rapid processing capabilities. Our observations indicate that our model outperforms other time series analysis methodologies, including LSTM model [14].

However, the feature selection method based on XGBoost's Gain values may retain highly correlated indicators such as volatility_dcl and volatility_bbh, potentially affecting interpretability. To improve the model, future studies could integrate statistical tests like VIF and domain-knowledge-driven feature synthesis to mitigate redundancy while preserving predictive power.

In conclusion, the model achieves commendable accuracy in price prediction and is applicable in real-world stock market scenarios, provided with relevant news and stock price data. As more powerful models are developed, we are looking forward to further optimizing the model to enhance training speed and robustness. Additionally, we are considering the incorporation of supplementary factors into stock price prediction, such as external shocks, which include news related to other corporations pertinent to the target stock.

Table 2: Hyperparameter combinations obtained by geneticalgorithm.

Hyperparameter:	Units	Time step	Learning rate	Batch size	Dropout
Value:	159	5	0.009	8	0.3

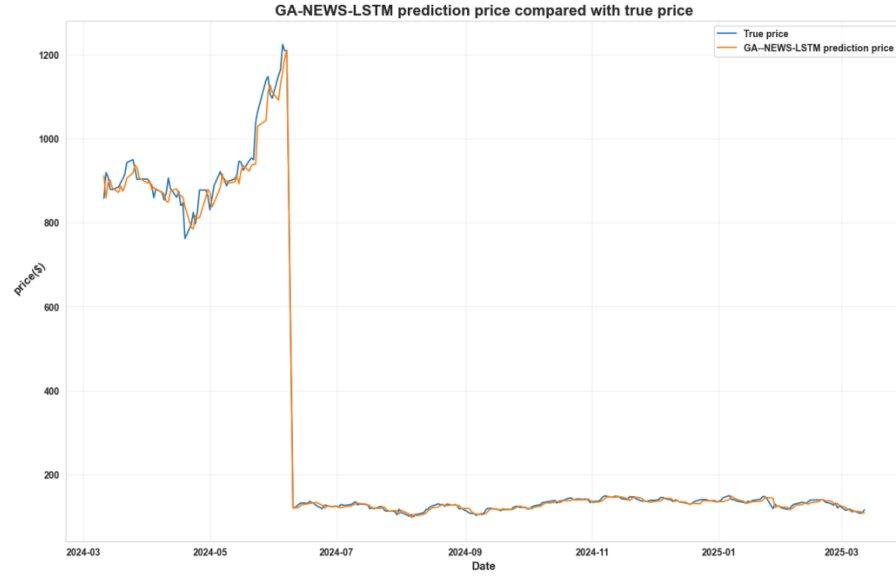
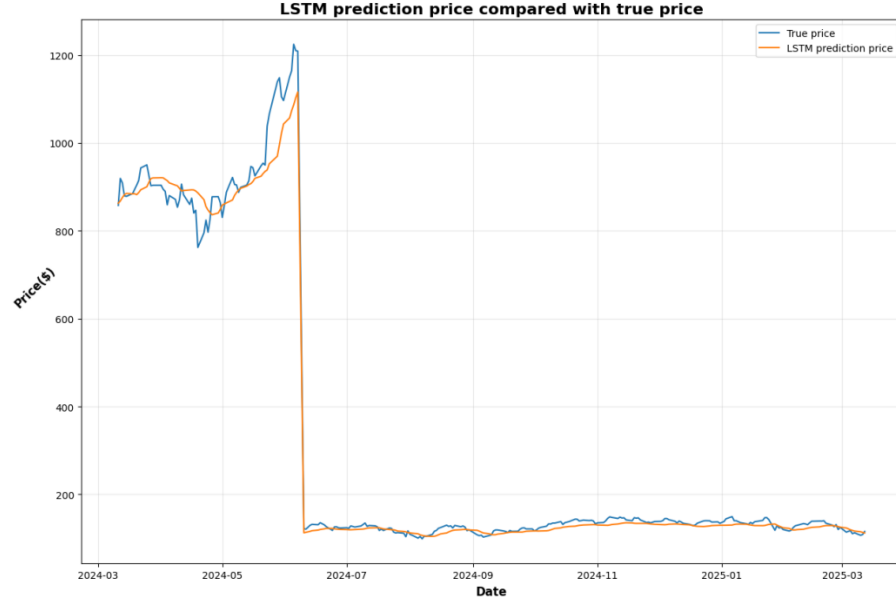
**Figure 4: GA-NEWS-LSTM prediction prices compared with true prices.****Figure 5: LSTM prediction prices compared with true prices.**

Table 3: Indicators comparison between GA-NEWS-LSTM and LSTM.

model	indicators	value
GA-NEWS-LSTM	R^2	0.99778
	MSE	276.4
	MAE	8.79535
LSTM	R^2	0.9854
	MSE	1748.8
	MAE	24.5

References

- [1] Artificial Intelligence Applied to Stock Market Trading: A Review. IEEE Xplore, 2021.
- [2] S. Selvin, R. Vinayakumar, E. A. Gopalakrishnan, V. K. Menon and K. P. So-man, "Stock price prediction using LSTM, RNN and CNN-sliding window model," 2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI), Udupi, India, 2017, pp. 1643-1647, doi: 10.1109/ICACCI.2017.8126078.
- [3] M. G. Sousa, K. Sakiyama, L. d. S. Rodrigues, P. H. Moraes, E. R. Fernandes and E. T. Matsubara, "BERT for Stock Market Sentiment Analysis," 2019 IEEE 31st International Conference on Tools with Artificial Intelligence (ICTAI), Portland, OR, USA, 2019, pp. 1597-1601, doi: 10.1109/ICTAI.2019.00231.
- [4] X. Weng, X. Lin and S. Zhao, "Stock Price Prediction Based On Lstm And Bert," 2022 International Conference on Machine Learning and Cybernetics (ICMLC), Japan, 2022, pp. 12-17, doi: 10.1109/ICMLC56445.2022.9941293.
- [5] Financial Sentiment Analysis: A Survey. IEEE Transactions on Knowledge and Data Engineering, 2023.
- [6] Li, X., Xie, H., Song, Y., Zhu, S., Li, Q., & Wang, F. L. (2015). Does summarization help stock prediction? A news impact analysis. IEEE intelligent systems, 30(3), 26-34.
- [7] Loughran, T., & McDonald, B. (2011). When is a liability not a liability? Textual analysis, dictionaries, and 10-Ks. The Journal of finance, 66(1), 35-65.
- [8] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019, June). Bert: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers) (pp. 4171-4186).
- [9] Jing, N., Wu, Z., & Wang, H. (2021). A hybrid model integrating deep learning with investor sentiment analysis for stock price prediction. Expert Systems with Applications, 178, 115019.
- [10] Sousa, M. G., Sakiyama, K., de Souza Rodrigues, L., Moraes, P. H., Fernandes, E. R., & Matsubara, E. T. (2019, November). BERT for stock market sentiment analysis. In 2019 IEEE 31st international conference on tools with artificial intelligence (ICTAI) (pp. 1597-1601). IEEE.
- [11] Bao, W., Yue, J., & Rao, Y. (2017). A deep learning framework for financial time series using stacked autoencoders and long-short term memory. PloS one, 12(7), e0180944.
- [12] A Survey on the Application of Genetic Algorithms in Financial Time Series Forecasting. Journal of Financial Data Science, 2024.
- [13] Analytics Vidhya, "Sentiment Analysis using Python," Analytics Vidhya, Jul. 12, 2022. [Online]. Available: <https://www.analyticsvidhya.com/blog/2022/07/sentiment-analysis-using-python/>
- [14] Comparative Analysis of Machine Learning Models for Stock Price Prediction. International Journal of Financial Engineering, 2024.

APPENDIX

Appendix 1. Features Selecting Principles of XGBoost

XGBoost (eXtreme Gradient Boosting) is an efficient gradient boosting tree framework that gradually corrects the prediction error by integrating multiple weak learners (decision trees). XGBoost's feature importance selecting is based on the contribution of a feature when it is used to split a node in the tree model. This paper chooses Gain as the importance indicator.

Each tree of XGBoost aims to maximize the gain of the objective function when splitting a node. The objective function consists of

a loss function and a regularization term, given by Equation (25):

$$obj = \sum_{i=1}^n L(y_i, \hat{y}_i) + \sum_{k=1}^k \Omega(f_k) \quad (25)$$

where obj is the objective function; n is the number of samples; $L(y_i, \hat{y}_i)$ is the loss function, this paper use mean squared error which is given by Equation (22); k is the number of decision trees whose maximum sets to 500; $\Omega(f_k)$ is regularization which formula is given by Equation (26):

$$\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \|\omega\|^2 \quad (26)$$

where γ is the minimum loss reduction threshold for splitting a node that sets to 0; T is the number of leaf nodes in the tree whose maximum sets to 6; λ is the minimum loss reduction threshold for splitting a node that sets to 1; ω is the the weight vector of the leaf node or in another word the predicted value for each leaf node. When splitting a node in the tree, the formula of gain by Equation (27):

$$Gain = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma \quad (27)$$

where I is the all sample sets of the current node; I_L is the sample set of the left subtree after the split; I_R is the sample set of the right subtree after the split; g_i is the first-order gradient of the loss function for sample i ; h_i is the second-order gradient of the loss function for sample i ; λ and γ is as the same as in Equation (26).

Then, this paper obtains each feature importance scores showed in Table 1 using the sum of Gain when this feature is used as a split node in all trees.

Appendix 2. Detailed Formula and Risk Analysis of Technical Indicators

In 4.3, this paper has introduced the trend visual ichimoku a has risk in data leakage. The formula of this indicator is given by Equation (28):

$$A_t = \frac{T_t + K_t}{2} \quad (28)$$

where T_t is the Tenkan-sen and K_t is Kijun-sen. Both these two indicators are based on the historical data. However, in ichimoku formula, it will calculate the current A_t at time point t then shift the result forward 26 days as the upper boundary of the cloud layer for the next 26 days. This will cause model seeing the future data so this paper replace it with trend ichimoku a whose use past 26 days data to shift forward 26 days, given by Equation (29):

$$A_t = \frac{T_{t-26} + K_{t-26}}{2} \quad (29)$$

In equation (29), The calculation of A_t is based entirely on the historical 26-day data (from $t-26$ to $t-1$), and then the calculated result is used as the value of the t -th day's A_t .

And the Tenkan-sen is given by Equation (30):

$$T_t = \frac{\max(P_{high}, 9) + \min(P_{low}, 9)}{2} \quad (30)$$

where $\max(P_{high}, 9)$ is the highest price in the past 9 days; $\min(P_{low}, 9)$ is the lowest price in the past 9 days. And the Kijun-sen is given by Equation (31):

$$K_t = \frac{\max(P_{high}, 26) + \min(P_{low}, 26)}{2} \quad (31)$$

where $\max(P_{high}, 26)$ is the highest price in the past 26 days; $\min(P_{low}, 26)$ is the lowest price in the past 26 days. And the other 3 indicators this paper used are given below respectively.

The formula of volatility dcl (Donchian Channel Lower Band) is given by Equation (32):

$$DCL_t = \min(P_{low}, 20) \quad (32)$$

where $\min(P_{low}, 20)$ is the lowest price in the past 20 days.

The formula of the volatility bbh (Bollinger Bands Upper Band) is given by Equation (33):

$$\frac{1}{N} \sum_{i=t-N+1}^t P_{close,i} + 2 \times \sqrt{\frac{1}{N} \sum_{i=t-N+1}^t (P_{close,i} - \frac{1}{N} \sum_{i=t-N+1}^t P_{close,i})^2} \quad (33)$$

where P_{close} is the close price; N is the time period that sets to 20.

The formula of the others cr (Accumulation/Distribution Line) is given by Equation (34):

$$A/D_t = \sum \left[\frac{(P_{close} - P_{low}) - (P_{high} - P_{close})}{P_{high} - P_{low}} \times V \right] \quad (34)$$

where P_{close} is the close price; P_{low} is the lowest price; P_{high} is the highest price; V is the volume.